Portafolio

Materia : Desarrollo de Software Multiplataforma

Nombre : Angel Yael Cota Murillo

Fecha : Diciembre 03, 2025

INDICE

Conjuntos (Sets) 4
Diccionarios (Maps / HashMaps / Dictionaries) 7
Algoritmos de Ordenamiento 13
Colas (Queues) 17
Pilas (Stacks) 21
La Recursividad 34
Tipos de Datos Abstractos (TDA) 37
Listas enlazadas (dinámicas): simple, doble o circular 38
Árbol (Tree) 38

Conjuntos (Sets)

Un **conjunto** es una **estructura de datos** que almacena **elementos únicos y no ordenados**, es decir, no permite duplicados y no mantiene un orden específico. Los conjuntos son muy útiles para **operaciones matemáticas como unión, intersección y diferencia**, así como para **eliminar duplicados en colecciones de datos**.

Características de los conjuntos

1. **Elementos únicos:** No se permiten duplicados.
2. **No ordenados:** No tienen índices como listas o arreglos.
3. **Operaciones eficientes:** Permiten realizar rápidamente operaciones de pertenencia, unión, intersección y diferencia.
4. **Dinámicos:** Se puede agregar o eliminar elementos en cualquier momento.

Operaciones comunes en conjuntos

- **Agregar elemento:** Insertar un elemento en el conjunto.
- **Eliminar elemento:** Quitar un elemento del conjunto.
- **Unión:** Combinar dos conjuntos sin duplicados.
- **Intersección:** Obtener los elementos comunes entre dos conjuntos.
- **Diferencia:** Obtener elementos presentes en un conjunto pero no en otro.

Ejemplo práctico: Conjuntos en Java (HashSet) import

```
java.util.HashSet;


public class ConjuntosEjemplo {

public static void main(String[] args) {

HashSet<Integer> conjuntoA = new

HashSet<>();

    HashSet<Integer> conjuntoB = new HashSet<>();
```

```
    // Agregar elementos
conjuntoA.add(1);
conjuntoA.add(2);
conjuntoA.add(3);
conjuntoB.add(3);        conjuntoB.add(4);

    // Unión
    HashSet<Integer> union = new HashSet<>(conjuntoA);
union.addAll(conjuntoB);
    System.out.println("Unión: " + union);  // Salida: [1, 2, 3, 4]

    // Intersección
    HashSet<Integer> interseccion = new HashSet<>(conjuntoA);
interseccion.retainAll(conjuntoB);
    System.out.println("Intersección: " + interseccion);  // Salida: [3]

    // Diferencia
    HashSet<Integer> diferencia = new HashSet<>(conjuntoA);
diferencia.removeAll(conjuntoB);
    System.out.println("Diferencia: " + diferencia);  // Salida: [1, 2]
  }
```

Ilustración conceptual de conjuntos

Conjunto A = {1, 2, 3, 4}

Conjunto B = {3, 4, 5, 6}

Unión: {1, 2, 3, 4, 5, 6}

Intersección: {3, 4}

Diferencia A-B: {1, 2} Diferencia

B-A: {5, 6}

Aplicaciones de los conjuntos

1. **Eliminación de duplicados** en listas o arreglos.
2. **Operaciones matemáticas** entre grupos de elementos.
3. **Búsquedas rápidas** de pertenencia.
4. **Algoritmos de teoría de grafos** (nodos conectados).
5. **Filtrado de datos** en bases de datos y sistemas de información.

Ventajas

- Garantiza que los elementos sean únicos.
- Operaciones como unión, intersección y diferencia son eficientes.
- Fácil de implementar y entender.

Desventajas

- No mantiene un orden específico de los elementos.
- No permite acceso por índice.
- Requiere más memoria que una lista simple si se manejan grandes cantidades de datos.

Referencias (Formato APA con enlace)

Goodrich, M., Tamassia, R., & Goldwasser, M. (2014). *Data Structures and Algorithms in Java*. Wiley.
Enlace: https://www.wiley.com/en-us/Data+Structures+and+Algorithms+in+Java%2C+6th+Edition-p-9781118771334

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
Enlace: https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/

Downey, A. (2015). *Think Python: How to Think Like a Computer Scientist*. Green Tea Press.
Enlace: https://greenteapress.com/wp/think-python-2e/

Diccionarios (Maps / HashMaps / Dictionaries)

Un **diccionario** es una estructura de datos que permite almacenar **pares clave-valor**, donde cada clave es **única** y se asocia a un valor.
Los diccionarios son ideales cuando se necesita **acceso rápido a datos mediante una clave**, en lugar de una posición numérica como en los arreglos o listas.

Características de los diccionarios

1. **Almacenamiento por clave:** Cada elemento se identifica mediante una **clave única**.
2. **Acceso rápido:** Permite buscar valores de manera eficiente usando la clave.
3. **Dinámico:** El tamaño puede aumentar o disminuir durante la ejecución.
4. **Valores heterogéneos:** Cada valor puede ser de cualquier tipo de dato, incluyendo listas, objetos u otros diccionarios.

Ejemplo práctico: Diccionario en Java (HashMap) import

```
java.util.HashMap;

public class DiccionarioEjemplo {
public static void main(String[] args) {
    HashMap<String, String> estudiante = new HashMap<>();

    // Agregar elementos
estudiante.put("nombre", "Luis");
estudiante.put("edad", "22");
estudiante.put("carrera", "Ingeniería");

    // Acceder a un valor
    System.out.println("Nombre: " + estudiante.get("nombre"));

    // Modificar un valor
estudiante.put("edad", "23");        // Recorrer el
diccionario        for (String clave :
estudiante.keySet()) {
        System.out.println(clave + ": " + estudiante.get(clave));
    }
  }
}
```

Salida:

Nombre: Luis

nombre: Luis

edad: 23 carrera:

Ingeniería

Ilustración conceptual de un diccionario

Diccionario: estudiante

Clave | Valor -------

---------- nombre

| Ana edad | 21

carrera | Ingeniería

promedio | 9.5

Aplicaciones de los diccionarios

1. **Bases de datos en memoria:** Guardar registros asociados a una clave.
2. **Mapeo de datos:** Asociar nombres de variables a valores.
3. **Contadores y estadísticas:** Guardar frecuencia de palabras o eventos.
4. **Caches y almacenamiento rápido de datos.**

Ventajas

- Acceso rápido a los elementos mediante la clave.
- Dinámico y flexible para almacenar distintos tipos de datos.
- Facilita operaciones de búsqueda, inserción y eliminación.

Desventajas

- Mayor uso de memoria debido a la gestión de claves.
- No mantiene un orden específico de los elementos (salvo diccionarios ordenados).
- Claves deben ser únicas y hashables.

Referencias (Formato APA con enlace)

Goodrich, M., Tamassia, R., & Goldwasser, M. (2014). *Data Structures and Algorithms in Java*. Wiley.
Enlace: https://www.wiley.com/en-us/Data+Structures+and+Algorithms+in+Java%2C+6th+Edition-p-9781118771334

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
Enlace: https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/

Downey, A. (2015). *Think Python: How to Think Like a Computer Scientist*. Green Tea Press.
Enlace: https://greenteapress.com/wp/think-python-2e/

Árboles (Trees)

Un **árbol** es una estructura de datos jerárquica compuesta por **nodos**, donde cada nodo contiene un **dato** y referencias a sus **hijos**. Se utiliza para representar relaciones **padre-hijo**, como en sistemas de archivos, jerarquías empresariales y expresiones matemáticas.

- El **nodo raíz** es el nodo principal del árbol.
- Los **nodos hoja** son aquellos que no tienen hijos.
- Los **nodos internos** tienen al menos un hijo.

Características de los árboles

1. **Jerarquía:** Los elementos están organizados de manera jerárquica.
2. **Unicidad de caminos:** Existe un único camino desde la raíz hasta cualquier nodo.
3. **Subárboles:** Cada nodo puede considerarse la raíz de un subárbol.
4. **Profundidad y altura:** La profundidad indica la distancia desde la raíz hasta un nodo, y la altura indica la distancia desde un nodo hasta la hoja más lejana.

Tipos de árboles

1. **Árbol binario:** Cada nodo tiene como máximo **dos hijos** (izquierdo y derecho).
2. **Árbol binario de búsqueda (BST):** Los nodos se organizan de manera que **el hijo izquierdo < nodo < hijo derecho**.
3. **Árbol AVL:** BST autoequilibrado que mantiene el balance de altura.
4. **Árbol rojo-negro:** BST con propiedades adicionales de balance.

Ejemplo práctico: Árbol binario en Java

```
// Clase Nodo class

Nodo {    int dato;

   Nodo izquierdo, derecho;


   public Nodo(int item) {

dato = item;       izquierdo =

derecho = null;

   }
```

```
}

// Clase Árbol class
ArbolBinario {
   Nodo raiz;

   void recorrerEnOrden(Nodo nodo) {
      if (nodo != null) {
recorrerEnOrden(nodo.izquierdo);
System.out.print(nodo.dato + " ");
recorrerEnOrden(nodo.derecho);
      }
   }
}

// Uso public class Principal {     public
static void main(String[] args) {
ArbolBinario arbol = new
ArbolBinario();        arbol.raiz = new
Nodo(10);        arbol.raiz.izquierdo =
new Nodo(5);        arbol.raiz.derecho =
new Nodo(15);

      System.out.println("Recorrido en orden:");
arbol.recorrerEnOrden(arbol.raiz); // Salida: 5 10 15
   }
}
```

Ilustración conceptual de un árbol binario

```
      10
     /  \
```

5 15

/ \ / \

2 7 12 20

- **Raíz:** 10
- **Hojas:** 2, 7, 12, 20
- **Subárbol izquierdo:** 5, 2, 7
- **Subárbol derecho:** 15, 12, 20

Aplicaciones de los árboles

1. Representación de **jerarquías** (directorios, organización empresarial).
2. Algoritmos de **búsqueda** (BST, AVL, árboles B).
3. Compiladores: **expresiones matemáticas** y sintaxis de programas.
4. Bases de datos y sistemas de archivos: **árboles B y B+**.

Ventajas

- Permiten **búsquedas rápidas**.
- Representan relaciones jerárquicas de manera natural.
- Base para estructuras complejas como **heaps** y **árboles de decisión**.

Desventajas

- Implementación más compleja que listas o arreglos.
- Requieren memoria adicional para referencias a hijos.
- Desbalance en árboles simples puede afectar el rendimiento.

Referencias (Formato APA con enlace)

Goodrich, M., Tamassia, R., & Goldwasser, M. (2014). *Data Structures and Algorithms in Java*. Wiley.
Enlace: https://www.wiley.com/en-us/Data+Structures+and+Algorithms+in+Java%2C+6th+Edition-p-9781118771334

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
Enlace: https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/

Algoritmos de Ordenamiento

Los **algoritmos de ordenamiento** son métodos que permiten **reorganizar los elementos de una colección de datos** (como un arreglo o lista) siguiendo un criterio específico, generalmente de menor a mayor o de mayor a menor. El ordenamiento es fundamental en programación, ya que facilita **búsquedas eficientes, procesamiento de datos y organización lógica**.

Tipos de algoritmos de ordenamiento

1. Ordenamiento de burbuja (Bubble Sort)

Es un algoritmo simple que compara elementos adyacentes y los intercambia si están en el orden incorrecto. Se repite hasta que no haya más intercambios. **Pseudocódigo**

```
for i = 0 to n-1:     for j = 0 to n-
i-1:        if arreglo[j] >
arreglo[j+1]:
          intercambiar(arreglo[j], arreglo[j+1])
```

Ejemplo en Java

```
int[] arr = {5, 2, 9, 1, 5}; for (int i = 0; i
< arr.length-1; i++) {     for (int j = 0; j
< arr.length-i-1; j++) {        if (arr[j] >
arr[j+1]) {            int temp = arr[j];
arr[j] = arr[j+1];            arr[j+1] =
temp;
       }
   }
}
```

Salida: [1, 2, 5, 5, 9]

2. Ordenamiento por selección (Selection Sort)

Selecciona el elemento mínimo en cada iteración y lo coloca en su posición correcta.

Pseudocódigo

```
for i = 0 to n-1:     min_index = i     for j
= i+1 to n:        if arreglo[j] <
arreglo[min_index]:
          min_index = j
intercambiar(arreglo[i], arreglo[min_index])
```

Ejemplo en Python

```
arr = [64, 25, 12, 22, 11] for
i in range(len(arr)):
   min_idx = i     for j in
range(i+1, len(arr)):        if
arr[j] < arr[min_idx]:
          min_idx = j     arr[i], arr[min_idx] =
arr[min_idx], arr[i]


print(arr)  # Salida: [11, 12, 22, 25, 64]
```

3. Ordenamiento por inserción (Insertion Sort)

Construye el arreglo ordenado **insertando cada elemento en su posición correcta** dentro de los elementos ya ordenados.

Pseudocódigo

```
for i = 1 to n-1:
key = arreglo[i]
   j = i-1
   while j >= 0 and arreglo[j] > key:
      arreglo[j+1] = arreglo[j]
      j = j-1
   arreglo[j+1] = key
```

Ejemplo en Java

```
int[] arr = {12, 11, 13, 5, 6}; for
(int i = 1; i < arr.length; i++) {
int key = arr[i];     int j = i - 1;
while (j >= 0 && arr[j] > key) {
arr[j + 1] = arr[j];        j = j - 1;
   }
   arr[j + 1] = key;
}
```

Salida: [5, 6, 11, 12, 13]

4. Ordenamiento rápido (Quick Sort)

Es un algoritmo eficiente basado en **divide y vencerás**, que selecciona un pivote y divide los elementos en menores y mayores que el pivote, ordenando recursivamente cada subarreglo. **Ejemplo en Python** def quicksort(arr):   if len(arr) <= 1:   return arr   pivot = arr[len(arr)//2] left = [x for x in arr if x < pivot]   middle = [x for x in arr if x == pivot]   right = [x for x in arr if x > pivot]   return quicksort(left) + middle + quicksort(right)

arr = [3,6,8,10,1,2,1] print(quicksort(arr)) #

Salida: [1, 1, 2, 3, 6, 8, 10]

Ilustración conceptual del Bubble Sort

Antes: 5 2 9 1 5

Iteración 1: 2 5 1 5 9

Iteración 2: 2 1 5 5 9 Iteración

3: 1 2 5 5 9 **Aplicaciones de**

los algoritmos de

ordenamiento

- Preparar datos para **búsquedas rápidas**.
- Ordenar listas de clientes, productos o registros.
- Mejorar eficiencia en algoritmos que dependen de datos ordenados, como **Búsqueda Binaria**.

Referencias (Formato APA con enlace)

Goodrich, M., Tamassia, R., & Goldwasser, M. (2014). *Data Structures and Algorithms in Java*. Wiley.
Enlace: https://www.wiley.com/en-us/Data+Structures+and+Algorithms+in+Java%2C+6th+Edition-p-9781118771334

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
Enlace: https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/

Downey, A. (2015). *Think Python: How to Think Like a Computer Scientist*. Green Tea Press.
Enlace: https://greenteapress.com/wp/think-python-2e/

Colas (Queues)

Una **cola** es un **Tipo de Dato Abstracto (TDA)** que permite almacenar elementos de manera **ordenada**, siguiendo el principio **FIFO (First In, First Out)**, es decir, **el primer elemento en entrar es el primero en salir**. Las colas son muy útiles en programación para **gestionar procesos en orden, tareas de impresión, buffers de datos y sistemas de atención al cliente**.

Características de una cola

1. **Acceso restringido:** Solo se puede acceder al **frente** de la cola para eliminar elementos y al **final** para insertar nuevos elementos.
2. **Operaciones básicas:**
   - **enqueue(x):** Agrega un elemento al final de la cola. ○

     dequeue(): Elimina el elemento del frente.
   - **peek() o front():** Devuelve el elemento del frente sin eliminarlo.
3. **Implementación:** Puede hacerse con **arreglos** o **listas enlazadas**.
4. **Uso de memoria:** Dinámica o estática según la implementación.

Ejemplo práctico: Cola en Java class Cola {

```
private int[] elementos;     private int
frente, finalCola, tamañoMax;


   public Cola(int tamaño) {
tamañoMax = tamaño;        elementos
= new int[tamañoMax];
       frente = 0;
       finalCola = 0;
   }


   public void enqueue(int valor) {
if (finalCola < tamañoMax) {
elementos[finalCola++] = valor;
       } else {
          System.out.println("Cola llena");
```

```
        }
    }

    public int dequeue() {        if
(frente < finalCola) {            return
elementos[frente++];
        } else {
            System.out.println("Cola vacía");
return -1;
        }
    }

    public int peek() {        if
(frente < finalCola) {
return elementos[frente];
        }
            return -1;
        }
    }
}

// Uso public class Principal {     public
static void main(String[] args) {
Cola cola = new Cola(5);
cola.enqueue(10);
cola.enqueue(20);
cola.enqueue(30);

        System.out.println("Frente de la cola: " + cola.peek());
        System.out.println("Elemento retirado: " + cola.dequeue());
        System.out.println("Nuevo frente: " + cola.peek());
```

```
    }
}
```

Salida esperada:

Frente de la cola: 10

Elemento retirado: 10

Nuevo frente: 20

Ilustración conceptual de una cola

Frente -> 10 -> 20 -> 30 -> Final

- **Inserción (enqueue):** Se agrega un nuevo elemento al final.
- **Eliminación (dequeue):** Se retira el elemento del frente.

Tipos de colas

1. **Cola simple:** FIFO básico.
2. **Cola circular:** El último elemento apunta al primero, optimizando el uso de memoria.
3. **Cola de prioridad:** Cada elemento tiene una prioridad; los elementos de mayor prioridad se atienden primero.
4. **Deque (doble extremo):** Permite insertar y eliminar elementos por ambos extremos.

Aplicaciones de las colas

- Gestión de **procesos en sistemas operativos**.
- Manejo de **impresoras y buffers de datos**.
- **Sistemas de atención al cliente** (tickets).
- Algoritmos de **búsqueda en grafos** (BFS).

Ventajas

- Mantiene el **orden de llegada** de los elementos.
- Fácil de implementar y entender.
- Base para estructuras más complejas como colas de prioridad.

Desventajas

- Acceso limitado a los elementos intermedios.
- Inserción y eliminación deben respetar el orden FIFO.
- En colas estáticas, tamaño limitado y posible desperdicio de memoria

Referencias (Formato APA con enlace)

Goodrich, M., Tamassia, R., & Goldwasser, M. (2014). *Data Structures and Algorithms in Java*. Wiley.
Enlace: https://www.wiley.com/en-us/Data+Structures+and+Algorithms+in+Java%2C+6th+Edition-p-9781118771334

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
Enlace: https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/

Downey, A. (2015). *Think Python: How to Think Like a Computer Scientist*. Green Tea Press.
Enlace: https://greenteapress.com/wp/think-python-2e/

Pilas (Stacks)

Una **pila** es un **Tipo de Dato Abstracto (TDA)** que permite almacenar elementos de manera **ordenada**, siguiendo el principio **LIFO (Last In, First Out)**, es decir, **el último elemento en entrar es el primero en salir**. Las pilas son fundamentales en programación para **manejar llamadas a funciones, deshacer operaciones, evaluaciones de expresiones y recorridos en estructuras de datos**.

Características de una pila

1. **Acceso restringido:** Solo se puede acceder al elemento que está en el tope.
2. **Operaciones básicas:**
   - **push(x):** Agrega un elemento al tope de la pila. ○ **pop():** Elimina el elemento del tope.
   - **peek() o top():** Devuelve el elemento del tope sin eliminarlo.
3. **Uso de memoria:** Puede implementarse con arreglos o listas enlazadas.
4. **Seguridad:** Permite un manejo controlado del acceso a los elementos.

Ejemplo :

```
class Pila {     private int[]
elementos;     private int
tope;     private int max;

   public Pila(int tamaño) {
max = tamaño;        elementos
= new int[max];
       tope = -1;
   }

   public void push(int valor) {
if (tope < max - 1) {
elementos[++tope] = valor;
       } else {
          System.out.println("Pila llena");
```

```
        }
    }

    public int pop() {        if (tope
>= 0) {            return
elementos[tope--];
        } else {
            System.out.println("Pila vacía");
return -1;
        }
    }

    public int peek() {        if
(tope >= 0) {            return
elementos[tope];
        } else {
            System.out.println("Pila vacía");
return -1;

        }
    }
}

// Uso public class Principal {     public
static void main(String[] args) {        Pila
pila = new Pila(5);        pila.push(10);
pila.push(20);        pila.push(30);

        System.out.println("Tope de la pila: " + pila.peek());
        System.out.println("Elemento retirado: " + pila.pop());
        System.out.println("Tope actual: " + pila.peek());
    }
```

}

Salida esperada:

Tope de la pila: 30

Elemento retirado: 30

Aplicaciones de las pilas

1. Manejo de **llamadas a funciones** en programación (pila de ejecución).
2. Algoritmos de **recursión**.
3. Evaluación de **expresiones matemáticas** (notación infija, posfija o prefija).
4. Implementación de **deshacer/rehacer** en editores de texto.
5. Navegadores web: historial de páginas visitadas.

Ventajas

- Acceso controlado al último elemento insertado.
- Fácil de implementar usando arreglos o listas enlazadas.
- Ideal para problemas que requieren reversión de operaciones.

Desventajas

- Acceso limitado a los elementos intermedios.
- Tamaño limitado si se implementa con arreglos estáticos.

Referencias (Formato APA con enlace)

Goodrich, M., Tamassia, R., & Goldwasser, M. (2014). *Data Structures and Algorithms in Java*. Wiley.
Enlace: https://www.wiley.com/en-us/Data+Structures+and+Algorithms+in+Java%2C+6th+Edition-p-9781118771334

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
Enlace: https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/

Downey, A. (2015). *Think Python: How to Think Like a Computer Scientist*. Green Tea Press.
Enlace: https://greenteapress.com/wp/think-python-2e/

Listas

Una **lista** es una estructura de datos que permite almacenar **una colección de elementos ordenados**, donde cada elemento tiene una **posición específica** y se pueden realizar operaciones como **inserción, eliminación y búsqueda**. A diferencia de los arreglos, las listas pueden ser **dinámicas**, es decir, su tamaño puede cambiar en tiempo de ejecución.

Tipos de listas

1. **Lista simple:** Cada nodo contiene un dato y un puntero al siguiente nodo.
2. **Lista doblemente enlazada:** Cada nodo contiene un dato, un puntero al siguiente nodo y otro al nodo anterior.
3. **Lista circular:** El último nodo apunta al primero, formando un ciclo.

Características de las listas

- Ordenadas: Los elementos mantienen un orden definido.
- Dinámicas: Se pueden agregar o eliminar elementos en cualquier posición.
- Se accede a los elementos mediante **nodos** y **punteros**, en lugar de índices fijos como en los arreglos.
- Pueden ser utilizadas para implementar pilas, colas y otras estructuras.

Ejemplo: //

Clase Nodo

```
class Nodo {
   int dato;
   Nodo siguiente;


   Nodo(int dato) {
      this.dato = dato;
this.siguiente = null;
   }
}
```

```
// Clase Lista class
Lista {
   Nodo cabeza;

   void agregarAlFinal(int valor) {
Nodo nuevo = new Nodo(valor);
      if (cabeza == null) {
cabeza = nuevo;
      } else {
         Nodo temp = cabeza;
while (temp.siguiente != null) {
temp = temp.siguiente;
         }
         temp.siguiente = nuevo;
      }
   }

   void mostrarLista() {
Nodo temp = cabeza;
while (temp != null) {
System.out.print(temp.dato
+ " -> ");          temp =
temp.siguiente;
      }
      System.out.println("null");
   }
}

// Uso de la lista public class Principal {
public static void main(String[] args) {
```

```
Lista lista = new Lista();

lista.agregarAlFinal(10);

lista.agregarAlFinal(20);

lista.agregarAlFinal(30);


    lista.mostrarLista(); // Salida: 10 -> 20 -> 30 -> null

  }

}
```

Ventajas de las listas

- Tamaño dinámico: No se necesita conocer el tamaño de antemano.
- Inserciones y eliminaciones eficientes en cualquier posición.
- Base para otras estructuras como pilas y colas.

Desventajas

- Acceso a elementos por índice más lento que en un arreglo.
- Mayor uso de memoria por los punteros adicionales.
- Operaciones de búsqueda secuencial pueden ser costosas.

Referencias (Formato APA con enlace)

Goodrich, M., Tamassia, R., & Goldwasser, M. (2014). *Data Structures and Algorithms in Java*. Wiley.
Enlace: https://www.wiley.com/en-us/Data+Structures+and+Algorithms+in+Java%2C+6th+Edition-p-9781118771334

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
Enlace: https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/

Clases Parametrizadas y Tipos Genéricos

En programación orientada a objetos, una **clase parametrizada** o **tipo genérico** es una clase que puede operar con **cualquier tipo de dato** sin necesidad de reescribir el código para cada tipo. Los **tipos genéricos** permiten escribir **código reutilizable, seguro y flexible**, garantizando que los tipos de datos sean consistentes en tiempo de compilación.

Conceptos clave

1. **Genéricos:**
   Permiten definir clases, interfaces o métodos con **parámetros de tipo**, que se especifican al instanciar la clase o llamar al método.
2. **Ventajas:**
   - Reutilización de código.
   - Seguridad de tipo (menos errores de conversión). ○ Facilita la creación de colecciones y estructuras de datos genéricas como listas, pilas, colas y árboles.

Ejemplo de uso: Clase genérica Caja

En Java

```
// Clase parametrizada con tipo T
class Caja<T> {     private T
contenido;

   public void set(T valor) {
contenido = valor;
   }

   public T get() {
      return contenido;
   }
}

public class Principal {     public static
void main(String[] args) {
```

```
        // Caja de enteros
        Caja<Integer> cajaInt = new Caja<>();
cajaInt.set(100);
        System.out.println("Contenido de cajaInt: " + cajaInt.get());


        // Caja de cadenas
        Caja<String> cajaString = new Caja<>();
cajaString.set("Hola Mundo");
        System.out.println("Contenido de cajaString: " + cajaString.get());
    }
}
```

Salida:

Contenido de cajaInt: 100

Contenido de cajaString: Hola Mundo

Ejemplo práctico: Lista genérica

Las colecciones genéricas permiten almacenar elementos de un tipo específico:

```
import java.util.ArrayList;
public class EjemploLista {     public
static void main(String[] args) {
        // Lista de enteros
        ArrayList<Integer> numeros = new ArrayList<>();
numeros.add(10);        numeros.add(20);
numeros.add(30);


        for (Integer n : numeros) {
            System.out.println(n);
        }


        // Lista de cadenas
```

```
        ArrayList<String> nombres = new ArrayList<>();
nombres.add("Ana");        nombres.add("Luis");


        for (String nombre : nombres) {
            System.out.println(nombre);
        }
    }
}
```

Ilustración:

Clase genérica: Caja<T>

```
+----------------+
|     Caja<T>    |
+----------------+
| - contenido:T  |
+----------------+
| + set(T valor) |
| + get(): T     |
+----------------+
```

Ejemplos de instanciación:

Caja<Integer> → contenido es Integer

Caja<String> → contenido es String

Tipos genéricos en métodos

No solo las clases pueden ser genéricas; los métodos también:

```
public <T> void imprimirArreglo(T[] arreglo) {
for (T elemento : arreglo) {
        System.out.println(elemento);
    }
}
```

Ventajas de los tipos genéricos

- Mayor **reutilización de código**.
- **Seguridad de tipos** en tiempo de compilación.
- Evita conversiones (*casting*) innecesarias.
- Facilita la creación de **colecciones personalizadas**.

Referencias (Formato APA con enlace)

Goodrich, M., Tamassia, R., & Goldwasser, M. (2014). *Data Structures and Algorithms in Java*. Wiley.
Enlace: https://www.wiley.com/en-us/Data+Structures+and+Algorithms+in+Java%2C+6th+Edition-p-9781118771334

Cay Horstmann, & Gary Cornell. (2013). *Core Java Volume I – Fundamentals*. Prentice Hall.
Enlace: https://www.pearson.com/store/p/core-java-volume-i-fundamentals/P100000964188

Downey, A. (2015). *Think Java: How to Think Like a Computer Scientist*. Green Tea Press.
Enlace: https://greenteapress.com/wp/think-java/

Arreglos (Arrays)

Un **arreglo** es una estructura de datos que permite almacenar **una colección de elementos del mismo tipo**, organizados en posiciones contiguas de memoria. Cada elemento se identifica mediante un **índice o posición**, que generalmente empieza en 0 en la mayoría de los lenguajes de programación.

Los arreglos son fundamentales porque permiten **acceder rápidamente a cualquier elemento** usando su índice, y son la base para otras estructuras de datos más complejas como listas, matrices y vectores.

Características de los arreglos

1. **Tamaño fijo:** La mayoría de los arreglos tienen un tamaño definido al momento de su creación.
2. **Tipo homogéneo:** Todos los elementos deben ser del mismo tipo (entero, cadena, booleano, etc.).
3. **Acceso rápido:** Se puede acceder a cualquier elemento mediante su índice en tiempo constante O(1).
4. **Almacenamiento contiguo:** Los elementos se guardan en posiciones consecutivas de memoria.

Ejemplo de declaración y uso

En Java

```
// Declaración de un arreglo de enteros int[]

numeros = new int[5];


// Inicialización de los elementos

numeros[0] = 10; numeros[1] =

20; numeros[2] = 30;

numeros[3] = 40; numeros[4]

= 50;


// Acceso a un elemento

System.out.println(numeros[2]);  // Imprime 30
```

```
// Recorrer el arreglo for(int i = 0; i <
numeros.length; i++){
   System.out.println(numeros[i]);
}
```

Tipos de arreglos

1. **Arreglos unidimensionales:** Una sola fila de elementos.
2. [10, 20, 30, 40, 50]
3. **Arreglos multidimensionales:** Como matrices (2D, 3D, etc.).
4. [[1, 2, 3],
5. [4, 5, 6],
6. [7, 8, 9]]
7. **Arreglos dinámicos:** Su tamaño puede cambiar en tiempo de ejecución (listas en Python, ArrayList en Java).

Ventajas de los arreglos

- Acceso rápido a elementos mediante índices.
- Eficientes para almacenar y recorrer datos.
- Base para estructuras más complejas (listas, colas, pilas, matrices).

Desventajas de los arreglos

- Tamaño fijo en muchos lenguajes.
- Operaciones como insertar o eliminar elementos en medio pueden ser costosas.
- Solo almacenan elementos del mismo tipo.

Ejemplo práctico: Suma de los elementos de un arreglo

En Java int suma = 0; for(int i = 0; i < numeros.length;

i++){ suma += numeros[i];

}

System.out.println("La suma es: " + suma);

Ilustración:

Índices: 0 1 2 3 4

Valores:10 20 30 40 50

Cada posición del arreglo permite acceder directamente al valor que contiene usando su índice.

Referencias (Formato APA con enlace)

Goodrich, M., Tamassia, R., & Goldwasser, M. (2014). *Data Structures and Algorithms in Java*. Wiley.
Enlace: https://www.wiley.com/en-us/Data+Structures+and+Algorithms+in+Java%2C+6th+Edition-p-9781118771334

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
Enlace: https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/

Downey, A. (2015). *Think Python: How to Think Like a Computer Scientist*. Green Tea Press.
Enlace: https://greenteapress.com/wp/think-python-2e/

La Recursividad

La **recursividad** es una técnica fundamental en programación que consiste en que una función se llama a sí misma para resolver un problema. Es especialmente útil cuando un problema grande puede dividirse en **subproblemas más pequeños del mismo tipo**.

Una función recursiva debe contener dos partes esenciales:

1. Caso base

Es la condición que detiene la recursión. Sin un caso base, la función se llamaría infinitamente, provocando un desbordamiento de pila (*stack overflow*).

2. Caso recursivo

Es la parte donde la función se llama a sí misma con un valor más pequeño o más simple, acercándose al caso base.

Ejemplo clásico: Factorial de un número

El factorial de un número entero positivo **n** (representado como **n!**) se define como:

- **n! = n × (n – 1)!**
- **1! = 1** (caso base)

Pseudocódigo:

func factorial(n): si n == 1

entonces retornar 1 si

no retornar n * factorial(n

- 1) **Ejemplo:** factorial(4)

→ 4 * factorial(3)

→ 4 * (3 * factorial(2))

→ 4 * (3 * (2 * factorial(1)))

→ 4 * 3 * 2 * 1

= 24

¿Dónde se usa la recursividad?

La recursividad se utiliza en múltiples áreas de las ciencias computacionales:

✓ Árboles y grafos

- Recorridos (preorden, inorden, postorden)
- Búsquedas (DFS)

✓ Ordenamientos

- QuickSort
- MergeSort

✓ Problemas matemáticos

- Secuencia de Fibonacci
- Torres de Hanoi

✓ Dividir y conquistar

- Resolver problemas reduciéndolos a versiones más pequeñas del mismo problema.

Ilustración :

```
factorial(4) |
 ├── factorial(3)
|     ├── factorial(2)
|    |      ├── factorial(1)
|    |      └── 1
|    └── 2 * 1
└── 3 * 2 * 1
= 24
```

Ventajas de la recursividad • Código más limpio y fácil de leer.

- Simplifica problemas complejos.
- Ideal para estructuras jerárquicas.

Desventajas

- Puede ser menos eficiente que una solución iterativa.
- Requiere más memoria por el uso de la pila de llamadas.
- Si no se maneja bien el caso base, provoca errores.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
Enlace: https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/

Downey, A. (2015). *Think Python: How to Think Like a Computer Scientist*. Green Tea Press.
Enlace: https://greenteapress.com/wp/think-python-2e/

Tipos de Datos Abstractos (TDA)

Un **Tipo de Dato Abstracto (TDA)** es un modelo matemático que define un conjunto de datos y las operaciones permitidas sobre ellos **sin especificar su implementación interna**. Esto permite separar **la lógica (qué hace)** de **la implementación (cómo lo hace)**, facilitando el diseño modular y la reutilización del código.

Los TDA son fundamentales en programación porque permiten estructurar la información de manera eficiente y comprensible.

1. Pila (Stack)

Una **pila** es un TDA que funciona con el principio **LIFO (Last In, First Out)**, es decir: el último elemento en entrar es el primero en salir.

Operaciones básicas

- **push(x):** Inserta un elemento.
- **pop():** Elimina el último elemento agregado.
- **peek():** Devuelve el elemento del tope sin eliminarlo.
-

Ejemplo en pseudocódigo:

pila = [] push(pila, 5)

push(pila, 7) pop(pila)

// elimina el 7

Ilustración

| 7 | ← Tope

| 5 |

---

2. Cola (Queue)

La **cola** trabaja bajo el principio **FIFO (First In, First Out)**, lo que significa que el primer elemento en entrar es el primero en salir.

Operaciones básicas

- **enqueue(x):** Inserta un elemento al final.
- **dequeue():** Elimina el elemento del frente.
-

Ejemplo:

cola = [] enqueue(cola, 4)

enqueue(cola, 8)

dequeue(cola) // elimina el 4

Ilustración

Frente → [4][8] → Final

3. Lista

Una **lista** es una colección ordenada de elementos que permite insertar, eliminar y acceder a datos en posiciones específicas.

Tipos de listas

- **Arreglos (listas estáticas)**
- **Listas enlazadas (dinámicas):** simple, doble o circular

Ejemplo:

lista = [10, 20, 30] insertar(lista,

15, pos=1)

4. Árbol (Tree)

Un **árbol** es una estructura jerárquica compuesta por nodos, donde cada nodo puede tener cero o más hijos.

Tipos comunes

- Árbol binario
- Árbol binario de búsqueda (BST)
- Árbol AVL
- Árbol rojo-negro

Ejemplo:

```
   10
  / \
 5   20
```

5. Grafo (Graph)

Un **grafo** está formado por **nodos (vértices)** y **aristas**, que representan relaciones entre ellos.

Tipos

- **Dirigido** y **no dirigido**
- **Con peso** y **sin peso**

Ejemplo:

```
A ---- B
 \    |
  \   |
    C
```

Referencias:

Goodrich, M., Tamassia, R., & Goldwasser, M. (2014). *Data Structures and Algorithms in Java*. Wiley.
Enlace:https://www.wiley.com/enus/Data+Structures+and+Algorithms+in+Java%2C+6th+Edition-p-9781118771334

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
Enlace: https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/